

MQTT: El Protocolo de Mensajería para IoT

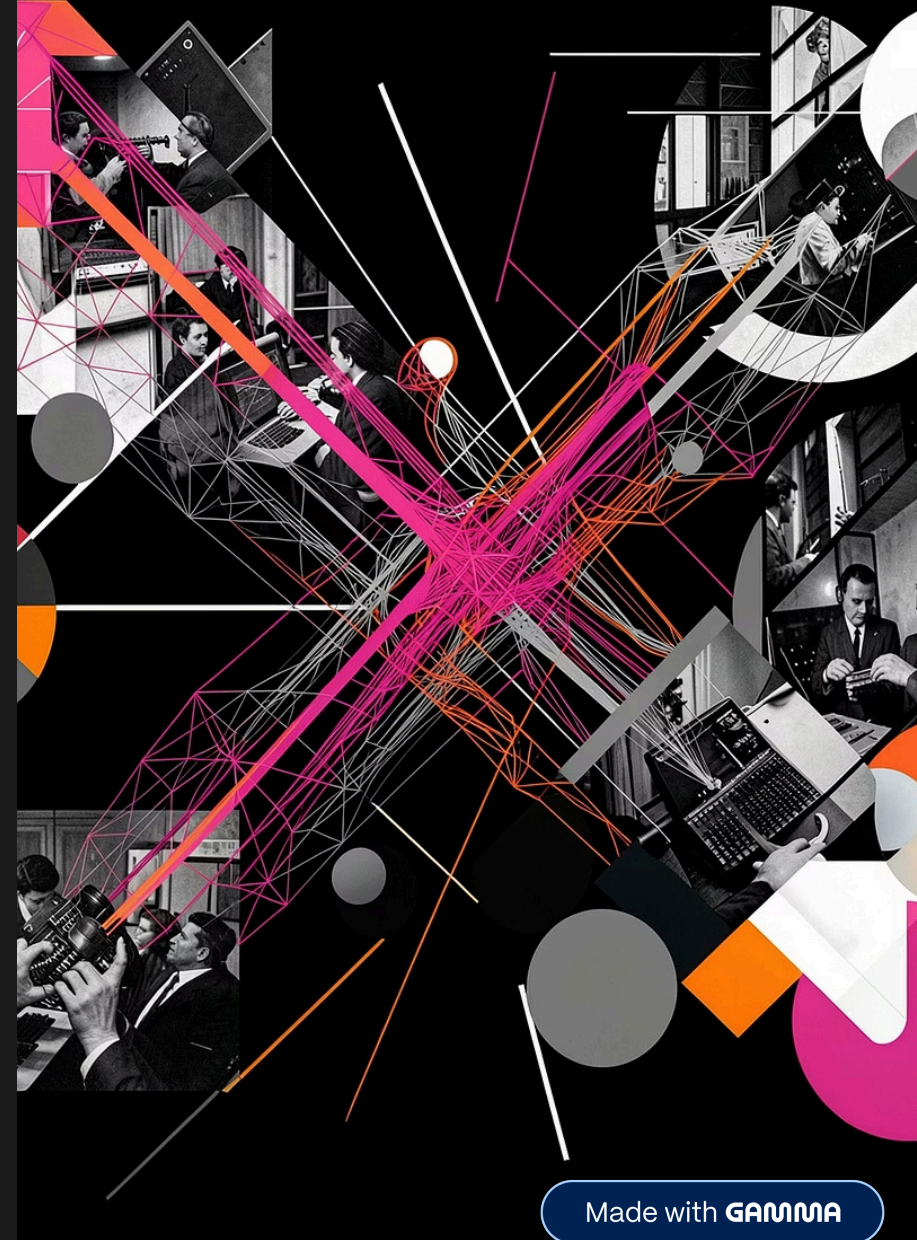
Message Queuing Telemetry Transport — ligero, eficiente y diseñado para el mundo conectado. Desde sensores hasta la nube, MQTT es el estándar de comunicación en tiempo real para dispositivos con recursos limitados.

- Samuel Moncada Salazar
- Alejandro Tabarez Rodriguez
- Julian Camilo Campo Cabrera

IOT

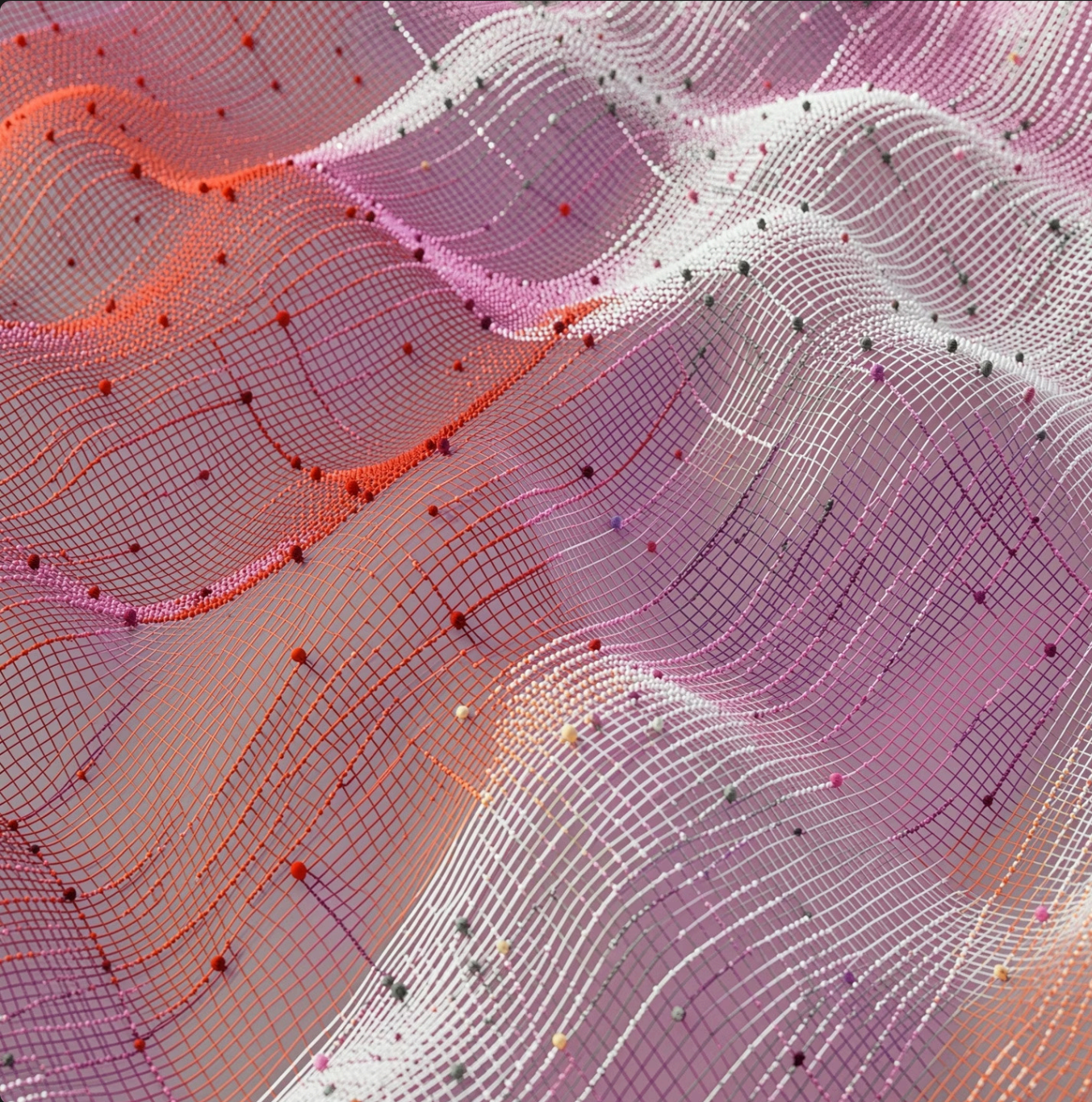
PROTOCOLO

PUBLISH/SUBSCRIBE



Made with GAMMA

¿Qué es MQTT?



Origen y propósito

MQTT fue creado por Andy Stanford-Clark y Arlen Nipper para conectar dispositivos con poca potencia en redes inestables. Hoy está mantenido por el estándar OASIS MQTT.

¿Dónde se usa?

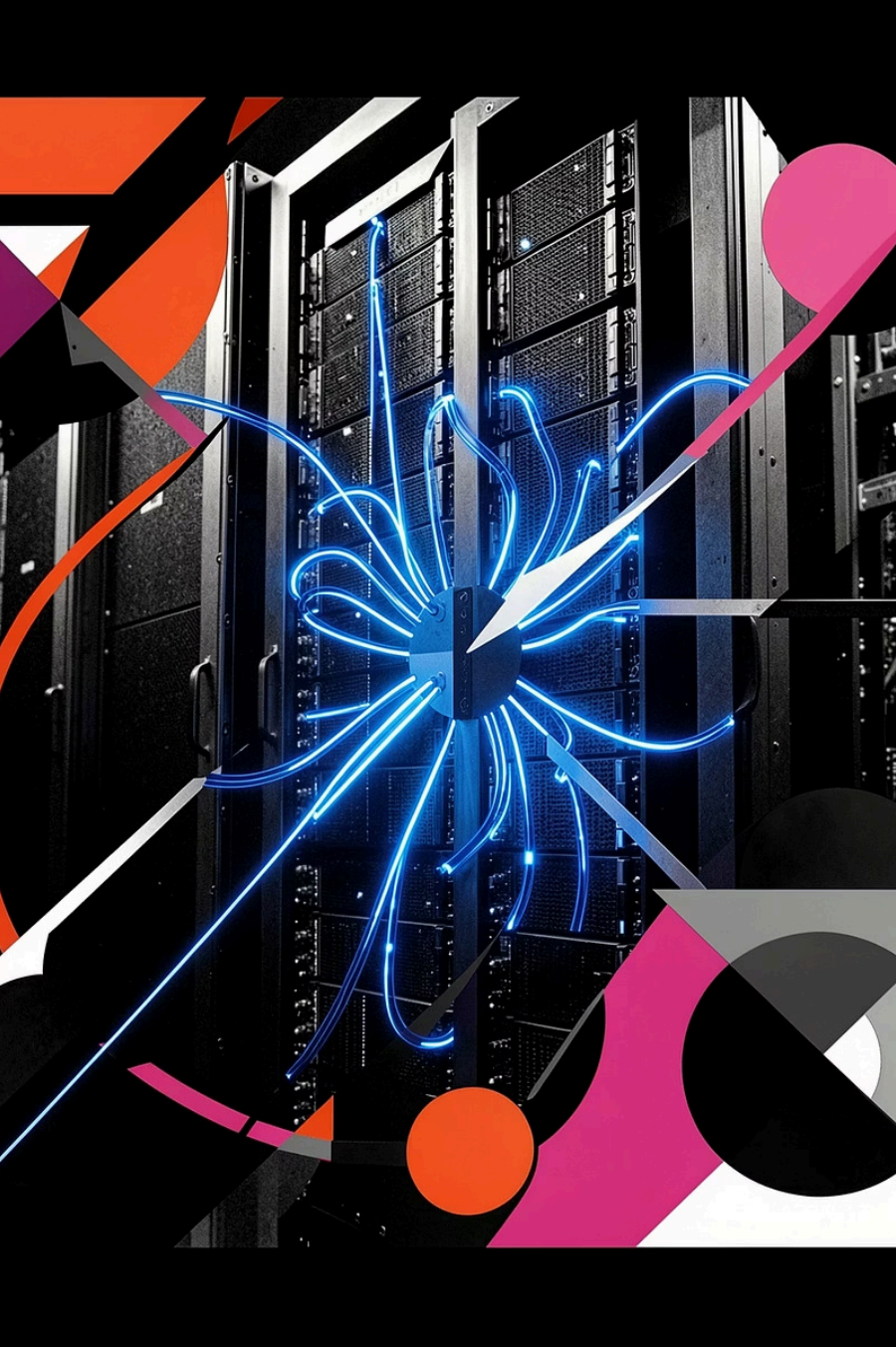
- Domótica y casas inteligentes
- Sensores industriales y SCADA
- Telemetría vehicular y GPS
- Agricultura de precisión
- Plataformas cloud IoT (AWS, Azure, GCP)

Arquitectura Publish/Subscribe

En lugar de que los dispositivos se comuniquen directamente, un **broker central** distribuye los mensajes a quienes estén suscritos a cada tema.



Este modelo desacopla emisores y receptores, mejorando la escalabilidad y la tolerancia a fallos del sistema.



Los Tres Componentes Clave

Cliente MQTT

Publica y/o recibe mensajes. Puede ser un sensor, ESP32, Raspberry Pi, app móvil o servidor.

Broker MQTT

El centro del sistema. Recibe mensajes y los reenvía a los suscriptores correctos. Ejemplos: Mosquitto, HiveMQ, EMQX.

Topics (Temas)

Rutas jerárquicas que organizan los mensajes: `casa/sala/temperatura`, `fabrica/maquina1/estado`. Soportan wildcards `+` y `#`.

Calidad de Servicio (QoS)

MQTT ofrece tres niveles de confiabilidad según las necesidades del sistema.

1

QoS 0 — At most once

Sin confirmación. El mensaje puede perderse, pero es el más rápido. Ideal para sensores frecuentes y datos no críticos.

2

QoS 1 — At least once

El mensaje llega al menos una vez. Requiere ACK. Puede duplicarse. Para automatización y notificaciones.

3

QoS 2 — Exactly once

Garantiza una sola entrega exacta. Más lento. Para pagos y comandos críticos.

Características Avanzadas

Retained Messages

El broker guarda el último mensaje de un topic. Los nuevos suscriptores lo reciben de inmediato (ej. estado de una luz).

Last Will & Testament

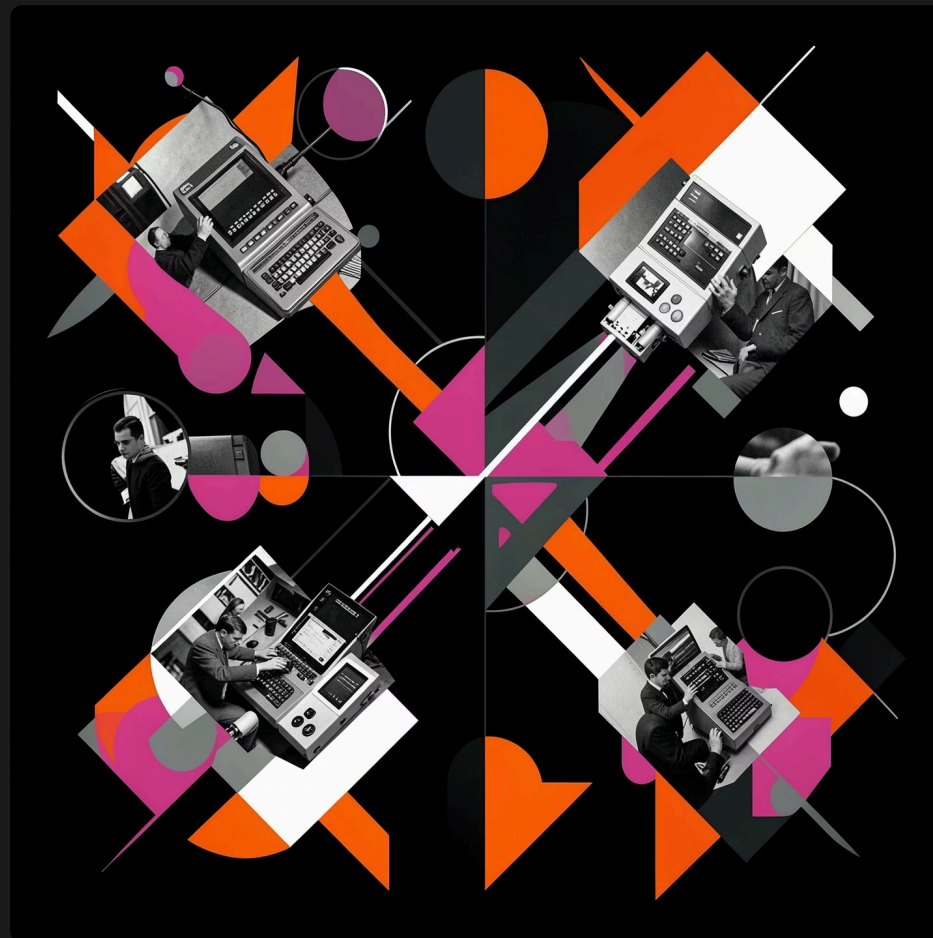
Mensaje automático si un cliente se desconecta inesperadamente. Clave para monitorización.

Keep Alive

Pings periódicos para mantener la conexión TCP activa. Si no llegan, el broker asume caída.

Wildcards

+ coincide con un nivel; # con todo lo restante. Ej: casa/+/temperatura, casa/#.



MQTT vs HTTP



¿Por qué MQTT para IoT?

HTTP fue diseñado para la web; MQTT fue diseñado para dispositivos con recursos limitados en redes inestables.

- Paquete mínimo de solo 2 bytes
- Conexión TCP persistente (sin reconectar)
- Comunicación bidireccional nativa
- Puertos: 1883 (sin cifrar) / 8883 (TLS)

Seguridad en MQTT

MQTT no cifra datos por defecto. La seguridad debe configurarse explícitamente en cada despliegue.



TLS/SSL

Cifra el canal de comunicación. Puerto seguro estándar: 8883.
Imprescindible en producción.



Autenticación

Usuario/contraseña, certificados X.509 o tokens JWT para verificar la identidad del cliente.



ACLs

Listas de control de acceso que definen qué topics puede leer o escribir cada cliente.



MQTT over WebSockets

Permite usar MQTT desde navegadores y dashboards web sin conexión TCP directa.

MQTT en Código: Ejemplos Prácticos

ESP32 / Arduino (C++)

```
client.publish("casa/sala/temp", "24");
```

Un ESP32 publica la temperatura en tiempo real. Muy usado con ESP8266, ESP32 y Raspberry Pi.

Python con Paho MQTT

```
import paho.mqtt.client as mqtt

client = mqtt.Client()
client.connect("broker.hivemq.com", 1883)
client.publish("test/demo", "Hola MQTT")
client.disconnect()
```

 **Brokers públicos para pruebas:** HiveMQ Public Broker y Eclipse Mosquitto Test Broker — ideales para experimentar sin infraestructura propia.

Python con Paho MQTT

- `pkg update`
- `pkg upgrade`
- `pkg install python`
- `pip install paho-mqtt`
- `pkg install termux-api`
- `nano bateria.py`

bateria.py

```
import ssl
import time
import json
import subprocess
import paho.mqtt.publish as publish
BROKER = "b9c04f87a5f64a61ac6c8666ed530332.s1.eu.hivemq.cloud"
PORT = 8883
TOPIC = "celular/bateria"
USUARIO = "samuel"
PASSWORD = "Samu2225"
while True:
    salida = subprocess.check_output(["termux-battery-status"]).decode("utf-8")
    datos = json.loads(salida)

    bateria = datos["percentage"]

    publish.single(
        TOPIC,
        f"{bateria}%",
        hostname=BROKER,
        port=PORT,
        auth={
            "username": USUARIO,
            "password": PASSWORD
        },
        tls={
            "tls_version": ssl.PROTOCOL_TLS_CLIENT
        }
    )

    print("Batería enviada:", bateria)

    time.sleep(5)
```

En el PC - receptor.py

```
import paho.mqtt.client as mqtt
from datetime import datetime
import ssl

BROKER = "b9c04f87a5f64a61ac6c8666ed530332.s1.eu.hivemq.cloud" # ← cambia esto
PORT = 8883
USUARIO = "samuel" # ← cambia esto
PASSWORD = "Samu2225" # ← cambia esto
TOPIC = "celular/bateria"

def on_connect(client, userdata, flags, rc, properties=None):
    if rc == 0:
        print("Conectado al broker")
        client.subscribe(TOPIC)
        print(f"Escuchando datos del celular en: {TOPIC}\n")
        print("-" * 45)
    else:
        print(f"Error de conexión, código: {rc}")

def on_message(client, userdata, msg):
    valor = msg.payload.decode()
    hora = datetime.now().strftime("%H:%M:%S")
    topic = msg.topic.split("/)[-1]

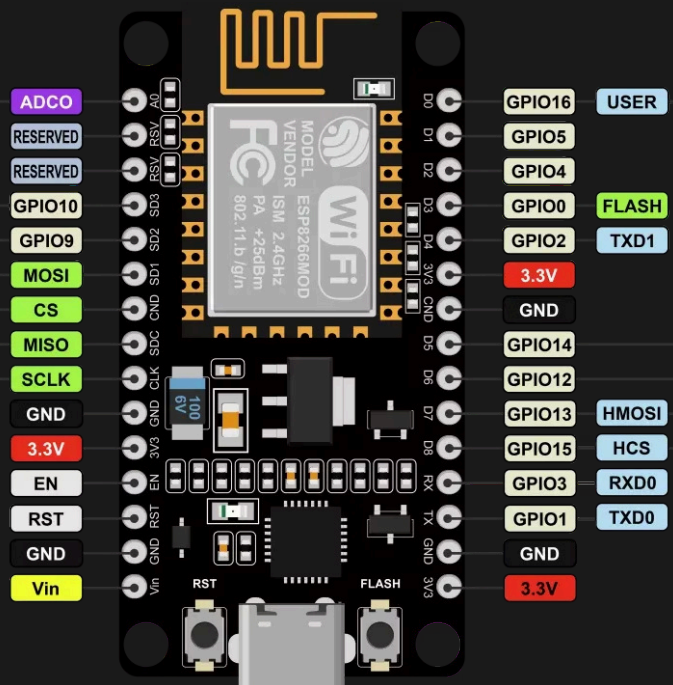
    print(f"[{hora}] {topic.upper()}: {valor}")

client = mqtt.Client(client_id="pc-receptor-01", callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
client.username_pw_set(USUARIO, PASSWORD)
client.tls_set(tls_version=ssl.PROTOCOL_TLS_CLIENT)

client.on_connect = on_connect
client.on_message = on_message

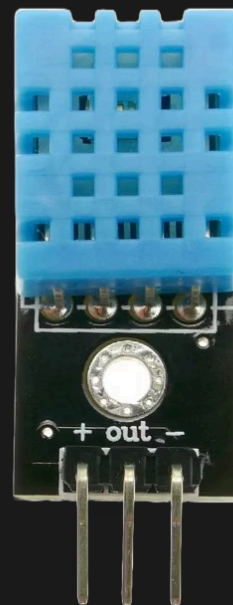
client.connect(BROKER, PORT)
client.loop_forever()
```

Materiales del ejercicio práctico



NodeMCU ESP8266

microcontrolador con WiFi integrado



Sensor DHT11 con módulo

mide temperatura y humedad



Computador con Mosquitto instalado

Arquitectura del ejercicio práctico



**ESP8266 +
DHT11**

Mide temperatura y
humedad



Publica
MQTT
puerto
1883



**Broker
Mosquitto**

Corre en el PC local
172.20.10.3



Distribuye a
suscriptores



mosquitto_sub

Suscriptor en terminal



Dashboard web

Gráficas en tiempo real

 El publicador y el broker están en la misma red WiFi local. El broker distribuye cada mensaje a todos los suscriptores simultáneamente.

¿Cómo suscribirte al ejercicio en vivo?

Linux



Conéctate al WiFi

Red: iPhone de Julian
Contraseña:
julian12345678



Instala el cliente MQTT

```
sudo apt install  
mosquitto-clients -y
```



Suscríbete al broker

```
mosquitto_sub -h 172.20.10.3 -t exposicion/# -v
```

Windows



Conéctate al WiFi

Red: iPhone de Julian
Contraseña:
julian12345678



Descarga e instala Mosquitto

mosquitto.org/download



Abre CMD y suscríbete

```
mosquitto_sub -h 172.20.10.3 -t exposicion/# -v
```

 Todos deben estar conectados a la misma red WiFi. Los mensajes llegarán cada 5 segundos con la temperatura y humedad del sensor.

Broker Mosquitto - dos puertos, dos protocolos



Puerto 1883 — MQTT estándar

Para el ESP8266

Protocolo TCP directo. El microcontrolador publica temperatura y humedad cada 5 segundos a través de este puerto. Es el puerto oficial del estándar MQTT.



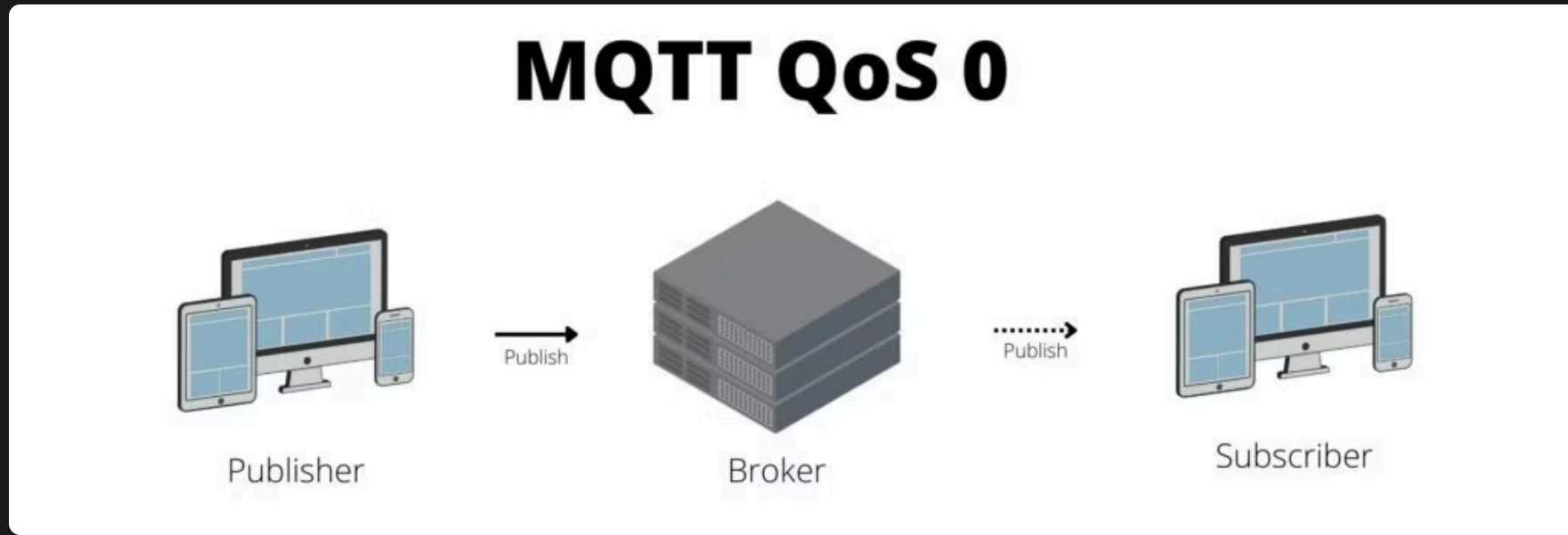
Puerto 9001 — MQTT sobre WebSocket

Para el dashboard web

Permite que un navegador web se conecte al broker directamente. Los navegadores no pueden usar TCP puro, pero sí WebSockets. El dashboard usa este puerto para recibir los datos en tiempo real.

⚠ La opción `allow_anonymous true` permite conexiones sin contraseña. En un entorno de producción real se configuraría usuario, contraseña y cifrado TLS.

¿Qué nivel de QoS usamos y por qué?



La barra o escala superior representa los tres niveles de Calidad de Servicio (QoS) disponibles en MQTT. Para nuestro ejercicio, hemos seleccionado QoS 0 por las razones que explicamos a continuación.

Nuestro ejercicio usa QoS 0

El sensor publica temperatura y humedad cada 5 segundos. Si un mensaje se pierde, el siguiente llega en 5 segundos. Los datos no son críticos: una lectura perdida no tiene consecuencias. QoS 0 es más rápido y consume menos recursos del ESP8266.

¿Cuándo usar QoS 2?

Apertura de una válvula de gas industrial. Activación de una alarma de incendio. Envío de una dosis de medicamento en equipos médicos. Cualquier comando que no puede ejecutarse ni una vez de más ni una de menos.

i La elección del nivel de QoS es una decisión de diseño que equilibra velocidad, consumo y confiabilidad.

